МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»
Кафедра інформаційних технологій

ЛАБОРАТОРНА РОБОТА №4

З дисципліни: «Об’єктно-орієнтоване та подійно-орієнтоване програмування»
На тему: «Робота з обліковими структурами даних»

Роботу виконав:
студент II курсу групи АД-251
спеціальності F6
Горбунова А.А.

Роботу перевірив:
к т.н., доцент Рудніченко М.Д.

Оцінка: _____

ДУОП, 2026

1 МЕТА РОБОТИ ТА ЗАВДАННЯ

1.1 Мета лабораторної роботи

Метою лабораторної роботи є формування стійких практичних навичок роботи з обліковими структурами даних та стандартними колекціями в мові програмування Java. У процесі виконання роботи необхідно:

– Ознайомитися з основними архітектурними видами колекцій Java (`List`, `Set`, `Map`);

– Навчитися проектувати динамічні списки, множини та асоціативні таблиці відображень;

– Опанувати базові операції додавання, пошуку, видалення та ітераційного перебору елементів колекцій;

– Практично застосувати механізм узагальнень (*Generics*) для забезпечення суворої типобезпеки на етапі компіляції додатка.

1.2 Загальне технічне завдання (Практична частина)

Програма досліджень вимагає розробки консольного додатка, що демонструє базові операції над трьома типами колекцій:

1) **Робота зі списками (`List<Student>`):** Створення динамічного списку об'єктів користувацького класу, виведення елементів за допомогою циклу `for-each`, розрахунок середнього арифметичного балу групи за допомогою методу `size()` та пошук лідера за успішністю.

2) **Робота з множинами (`Set<String>`):** Організація сховища унікальних навчальних дисциплін на базі `HashSet` з демонстрацією автоматичного відсікання дублікатів при спробі повторного додавання елементів.

3) **Робота з відображеннями (`Map<Integer, String>`):** Реалізація індексної таблиці відповідності між унікальним кодом (ідентифікатором) студента та

його ПІБ за допомогою `HashMap`. Демонстрація пошуку за ключем, перевірки наявності індексу та виведення записів через `entrySet()`.

1.3 Індивідуальне завдання (Варіант №7)

Розробити консольну програму для обліку фільмів. Створити користувацький клас `Movie`, який містить назву, жанр, рік випуску та поточний рейтинг кінострічки.

Для організації збереження даних необхідно використати динамічну колекцію `List<Movie>`. Програма повинна вивести повний список фільмів, розробити лінійний алгоритм визначення фільму з найвищим рейтингом у списку, а також автоматично сформувати та вивести множину унікальних жанрів кінематографа за допомогою структури `Set<String>`.

1.4 Оформити звіт і відповісти на контрольні питання.

2 ПЕРЕВІРКА ВИКОНАННЯ ЗАГАЛЬНОГО АВДАННЯ

У цьому розділі наведено детальний опис та аналіз практичної реалізації перших трьох пунктів програми досліджень, орієнтованих на освоєння базових інтерфейсів *Java Collections Framework*.

2.1 Робота зі списками об’єктів (`List<Student>`)

Згідно з пунктом 1 завдання, було спроектовано динамічний лінійний список на базі узагальненого класу `ArrayList<Student>` . На відміну від статичних масивів, дана структура дозволяє гнучко додавати нові об’єкти користувацького класу без ручного контролю ємності пам’яті.

В межах тестування було виконано такі операції:

– **Динамічне розширення:** За допомогою вбудованого методу `add()` до списку було послідовно додано трьох студентів із фіксованими анкетними даними.

– **Ітераційний аналіз даних:** За допомогою циклу `for-each` реалізовано прозорий перебір елементів колекції. Застосування методу `size()` дозволило динамічно розрахувати середній академічний бал групи, повністю нівелюючи ризики помилок «жорсткого кодування» лімітів масивів.

2.2 Робота з унікальними множинами (`Set<String>`)

Пункт 2 завдання присвячений дослідженню механізмів гарантії унікальності даних за допомогою інтерфейсу `Set` на прикладі реалізації `HashSet<String>`.

Для демонстрації захисту від появи дублікатів (аналог обмеження `UNIQUE` у реляційних СУБД) до колекції було виконано чотири спроби додавання назв навчальних дисциплін, причому ідентифікатор `"Java"` надсилався до сховища двічі. Результати тестування повністю підтвердили низькорівневу логіку роботи хеш-

таблиці: дублікат був автоматично відфільтрований за допомогою методів `hashCode()` та `equals()`, і у множині зберігся лише один унікальний екземпляр рядка.

2.3 Реалізація індексної таблиці відображень (`Map<Integer, String>`)

Згідно з пунктом 3 завдання, було розгорнуто асоціативну карту `HashMap<Integ String>`, яка виступає прямим архітектурним аналогом організації індексного пошуку в базах даних за **первинним ключем (Primary Key)**.

У створеній структурі як унікальний ключ (*Key*) використовується обгортка примітиву `Integer` (код студента), а як значення (*Value*) — об'єктний рядок `String` (ПІБ). Практичне тестування підтвердило константну швидкість доступу до даних за допомогою методу `get(102)`, а також продемонструвало ефективність повного перебору пар «ключ – значення» через ітераційну множину записів `entrySet()`.

3 ОПИС ПОБУДОВИ КОДУ ІНДИВІДУАЛЬНОГО ЗАВДАННЯ. ВАРІАНТ №7

subsectionОб’єктно-орієнтована декомпозиція та архітектура колекцій

Згідно з індивідуальним Варіантом №7, було розроблено об’єктно-орієнтовану програму для комплексного обліку та аналізу фільмів у кінокаталозі. На відміну від першої частини роботи, де операції виконувалися безпосередньо у методі `main()`, тут всю бізнес-логіку обробки контейнерів повністю інкапсульовано всередині окремого сервісного класу-менеджера.

Архітектура індивідуального завдання побудована на взаємодії таких компонентів:

1) **Клас сутності `Movie.java`:** Описує модель фільму, повністю приховуючи (інкапсулюючи) внутрішні поля назви (`title`), жанру (`genre`), року випуску (`year`) та рейтингу (`rating`) за допомогою модифікатора `private`. Доступ до атрибутів надається через публічні методи-геттери.

2) **Клас логіки `MovieManager.java`:** Виступає в ролі локальної бази даних предметої області. Всередині нього оголошено дві узагальнені структури даних: лінійний список `List<Movie>` для збереження хронології додавання елементів та хеш-множину `Set<String>` для накопичення жанрів.

3.1 Аналіз роботи узагальнених структур даних (*Generics*)

Впровадження колекцій *Java Collections Framework* дозволило вивести надійність та гнучкість коду індивідуального завдання на промисловий рівень за рахунок таких рішень:

– **Автоматична фільтрація унікальних атрибутів:** При додаванні нового об’єкта через метод `addMovie(Movie movie)`, менеджер не просто вставляє посилання у список, а паралельно виконує інструкцію `uniqueGenres.add(movie.g`
Це дозволяє динамічно формувати чисту множину унікальних жанрів кінематогра-

фа «на льоту», повністю позбавляючи розробника від написання громіздких вкладених циклів порівняння рядків.

– **Типобезпечний пошук лідера рейтингу:** Завдяки використанню суворих узагальнень (`List<Movie>`), компілятор Java гарантує, що у списку містяться виключно об’єкти фільмів. Це дозволило реалізувати швидкий лінійний алгоритм пошуку максимального рейтингу за допомогою методу `get(0)` та прямого виклику геттера `movie.getRating()` без використання ризикованих операцій приведення типів та оператора `instanceof`, які застосовувалися у минулих лабораторних роботах.

4 ЛІСТИНГ ПРОГРАМНОГО КОДУ

У цьому розділі наведено повний вихідний код. Код оформлено згідно зі стандартами *Oracle Code Conventions* та містить вичерпну документацію у форматі *JavaDoc*.

4.1 Клас Main.java

```
/*
 * Copyright (c) 2026. Anastasiia Gorbunova
 * National University "Odesa Polytechnic"
 *
 * Licensed under the Apache License, Version 2.0 (the "License");
 * you may not use this file except in compliance with the License.
 * You may obtain a copy of the License at
 *
 *     http://apache.org
 */
package ad251.gorbunova;

/* Імпорт стандартних колекцій Java Collections Framework*/
import java.util.ArrayList;    // Динамічний лінійний список студентів групи
import java.util.HashMap;      // Асоціативна хеш-мапа для індексного пошуку за кодом
import java.util.HashSet;      // Множина для збереження унікальних навчальних дисциплін
import java.util.List;         // Інтерфейс впорядкованих послідовностей даних
import java.util.Map;          // Інтерфейс для структур зберігання пар "ключ - значення"
import java.util.Set;          // Інтерфейс унікальних контейнерів без дублікатів

/**
 * Головний керуючий клас-оркестратор Лабораторної роботи №4.
 * Координує виконання спільної практичної частини та індивідуального Варіанту №7.
 *
 * @author Анастасія Горбунова
 * @version 4.0 15 Oct 2026
 */
public class Main {

    /**
     * Головна точка входу в додаток.
     *
     * @param args аргументи командного рядка CLI
     */
    public static void main(String[] args) {
        System.out.print("=== Лабораторна робота №4 ===\n");
        System.out.print("Тема: Робота з обліковими структурами даних\n\n");

        // ========================================================================
        // 1. ЗАГАЛЬНА ПРАКТИЧНА ЧАСТИНА (Робота зі списками Student через List)
        // ========================================================================
        System.out.print("--- 1. Тестування структури List<Student> ---\n");
        List<Student> students = new ArrayList<>();
        students.add(new Student("Олена Коваль", 20, 91.5));
        students.add(new Student("Андрій Петренко", 21, 87.2));
        students.add(new Student("Марія Шевченко", 19, 95.1));

        for (Student student : students) {
            student.printInfo();
        }

        double sum = 0;
        for (Student student : students) {
            sum += student.getAverageGrade();
        }
        System.out.printf("Середній бал навчальної групи: %.2f\n\n", (sum / students.size()));

        // ========================================================================
        // 2. ТЕСТУВАННЯ МНОЖИН (Set) - Демонстрація відсутності дублікатів
        // ========================================================================
        System.out.print("--- 2. Тестування структури Set<String> (Унікальність) ---\n");
        Set<String> subjects = new HashSet<>();
        subjects.add("Java");
```

```
64          subjects.add("Математика");
65          subjects.add("Бази даних");
66          subjects.add("Java"); /* Спроба силового додавання дубліката */
67
68          System.out.print("Навчальні дисципліни (дублікат відфільтровано): " + subjects + "\n\n");
69
70          // ========================================================================
71          // 3. ТЕСТУВАННЯ АСОЦІАТИВНИХ МАП (Map - Пошук за первинним ключем)
72          // ========================================================================
73          System.out.print("--- 3. Тестування структури Map<Integer, String> (Індекси) ---\n");
74          Map<Integer, String> studentMap = new HashMap<>();
75          studentMap.put(101, "Олена Коваль");
76          studentMap.put(102, "Андрій Петренко");
77          studentMap.put(103, "Марія Шевченко");
78
79          System.out.print("Отримання студента за кодом 102: " + studentMap.get(102) + "\n");
80          System.out.print("Повний перелік записів індексної таблиці Map:\n");
81          for (Map.Entry<Integer, String> entry : studentMap.entrySet()) {
82              System.out.print("  Код (Key): " + entry.getKey() + " -> Студент (Value): " +
                ↪  entry.getValue() + "\n");
83          }
84          System.out.print("\n");
85
86          // ========================================================================
87          // 4. ІНДИВІДУАЛЬНЕ ЗАВДАННЯ: ВАРІАНТ №7 (Облік фільмів)
88          // ========================================================================
89          System.out.print("--- 4. Виконання індивідуального Варіанту №7 (Облік фільмів) ---\n");
90          MovieManager manager = new MovieManager();
91
92          manager.addMovie(new Movie("Початок (Inception)", "Фантастика", 2010, 8.8));
93          manager.addMovie(new Movie("Інтерстеллар", "Драма", 2014, 8.7));
94          manager.addMovie(new Movie("Темний лицар", "Екшн", 2008, 9.0));
95          manager.addMovie(new Movie("Матриця", "Фантастика", 1999, 8.7));
96          manager.addMovie(new Movie("Кримінальне чтиво", "Кримінал", 1994, 8.9));
97
98          manager.printAllMovies();
99          manager.displayHighestRatedMovie();
100         manager.printGenres();
101     }
102 }
```

4.2 Клас Student.java

```java
/*
 * Copyright (c) 2026. Anastasiia Gorbunova
 * National University "Odesa Polytechnic"
 *
 * Licensed under the Apache License, Version 2.0 (the "License");
 * you may not use this file except in compliance with the License.
 * You may obtain a copy of the License at
 *
 *     http://apache.org
 */
package ad251.gorbunova;

/**
 * Клас, що описує сутність "Студент" для загальної практичної частини.
 * Забезпечує інкапсуляцію анкетних та успішних даних студента групи.
 *
 * @author Анастасія Горбунова
 * @version 4.0 15 Oct 2026
 */
public class Student {
    /** ПІБ студента */
    private final String name;
    /** Повний вік студента */
    private final int age;
    /** Середній академічний бал */
    private final double averageGrade;

    /**
     * Конструктор класу Student для повної ініціалізації полів.
     *
     * @param name ПІБ студента
     * @param age вік
     * @param averageGrade середній бал успішності
     */
    public Student(String name, int age, double averageGrade) {
        this.name = name;
        this.age = age;
        this.averageGrade = averageGrade;
    }

    /**
     * Виводить форматовану інформацію про поточний стан об'єкта у консоль.
     */
    public void printInfo() {
        System.out.printf("Студент: %-20s | Вік: %d | Середній бал: %.1f\n",
                          name,
                          age,
                          averageGrade);
    }

    // --- Публічні геттери доступу до приватних полів (Інкапсуляція) ---
    public String getName() { return name; }
    public int getAge() { return age; }
    public double getAverageGrade() { return averageGrade; }
}
```

4.3 Клас Movie.java

```java
/*
 * Copyright (c) 2026. Anastasiia Gorbunova
 * National University "Odesa Polytechnic"
 *
 * Licensed under the Apache License, Version 2.0 (the "License");
 * you may not use this file except in compliance with the License.
 * You may obtain a copy of the License at
 *
 *     http://apache.org
 */
package ad251.gorbunova;

/**
 * Клас, що описує сутність "Фільм" згідно з індивідуальним Варіантом №7.
 * Демонструє роботу об'єктних моделей усередині динамічних колекцій.
 *
 * @author Анастасія Горбунова
 * @version 4.0 15 Oct 2026
 */
public class Movie {
    /** Назва кінострічки */
    private final String title;
    /** Жанр фільму */
    private final String genre;
    /** Рік офіційного випуску */
    private final int year;
    /** Рейтинг фільму від 0.0 до 10.0 */
    private final double rating;

    /**
     * Конструктор для ініціалізації інкапсульованих даних фільму.
     *
     * @param title назва фільму
     * @param genre жанрова приналежність
     * @param year рік виходу на екрани
     * @param rating оцінка критиків
     */
    public Movie(String title, String genre, int year, double rating) {
        this.title = title;
        this.genre = genre;
        this.year = year;
        this.rating = rating;
    }

    /**
     * Форматує та виводить унікальні атрибути об'єкта фільму в консоль.
     */
    public void printInfo() {
        System.out.printf("Фільм: %-25s | Жанр: %-12s | Рік: %d | Рейтинг: %.1f\n",
                          title,
                          genre,
                          year,
                          rating);
    }

    // Публічні інкапсульовані методи доступу
    public String getTitle() { return title; }
    public String getGenre() { return genre; }
    public int getYear() { return year; }
    public double getRating() { return rating; }
}
```

4.4 Клас MovieManager.java

```java
/*
 * Copyright (c) 2026. Anastasiia Gorbunova
 * National University "Odesa Polytechnic"
 *
 * Licensed under the Apache License, Version 2.0 (the "License");
 * you may not use this file except in compliance with the License.
 * You may obtain a copy of the License at
 *
 *     http://apache.org
 */
package ad251.gorbunova;

/* Підключення системних утиліт та колекцій за стандартом Oracle */
import java.util.ArrayList;   // Реалізація динамічного списку на базі масиву
import java.util.HashSet;   // Реалізація хеш-множини для зберігання унікальних елементів
import java.util.List;      // Базовий інтерфейс для роботи зі впорядкованими списками
import java.util.Set;       // Базовий інтерфейс для роботи з унікальними множинами даних

/**
 * Сервісний менеджер-аналізатор колекцій об'єктів (індивідуальне завдання Варіант №7).
 * Використовує механізм узагальнень (Generics) для забезпечення типобезпеки.
 *
 * @author Анастасія Горбунова
 * @version 4.0 15 Oct 2026
 */
public class MovieManager {
    /** Динамічний лінійний список для збереження послідовності об'єктів */
    private final List<Movie> movieList = new ArrayList<>();
    /** Хеш-множина для автоматичного відсікання дублікатів та унікальних жанрів */
    private final Set<String> uniqueGenres = new HashSet<>();

    /**
     * Додає об'єкт фільму до списку та реєструє його жанр у унікальній множині.
     *
     * @param movie екземпляр класу Movie
     */
    public void addMovie(Movie movie) {
        movieList.add(movie);
        uniqueGenres.add(movie.getGenre());
    }

    /**
     * Проводить ітераційний перебір списку за допомогою циклу for-each.
     */
    public void printAllMovies() {
        System.out.print("Повний список кінофільмів у базі даних List:\n");
        for (Movie movie : movieList) {
            movie.printInfo();
        }
    }

    /**
     * Реалізує лінійний алгоритм пошуку об'єкта з максимальним рейтингом у List.
     */
    public void displayHighestRatedMovie() {
        if (movieList.isEmpty()) {
            return;
        }

        Movie bestMovie = movieList.get(0);
        for (Movie movie : movieList) {
            if (movie.getRating() > bestMovie.getRating()) {
                bestMovie = movie;
            }
        }
        System.out.print("\nФільм із найвищим рейтингом за Варіантом №7:\n  --> ");
        bestMovie.printInfo();
    }

    /**
     * Виводить унікальну хеш-таблицю жанрів.
     */
    public void printGenres() {
        System.out.print("\nМножина унікальних жанрів у базі Set:\n  "
                         + uniqueGenres + "\n");
    }
}
```

5 РЕЗУЛЬТАТИ РОБОТИ ПРОГРАМИ

Для повної перевірки працездатності розробленої об’єктно-орієнтованої системи, контролю унікальності даних у множинах та індексного доступу в асоціативних картах було здійснено запуск програми в єдиному циклі збирання середовища VS Code. На рисунку 5.1 представлено фінальний вивід результатів у термінал Windows PowerShell.

```
=== Лабораторна робота №4 ===
Тема: Робота з обліковими структурами даних

--- 1. Тестування структури List<Student> ---
Студент: Олена Коваль          | Вік: 20 | Середній бал: 91,5
Студент: Андрій Петренко       | Вік: 21 | Середній бал: 87,2
Студент: Марія Шевченко        | Вік: 19 | Середній бал: 95,1
Середній бал навчальної групи: 91,27

--- 2. Тестування структури Set<String> (Унікальність) ---
Навчальні дисципліни (дублікат відфільтровано): [Java, Математика, Бази даних]

--- 3. Тестування структури Map<Integer, String> (Індекси) ---
Отримання студента за кодом 102: Андрій Петренко
Повний перелік записів індексної таблиці Map:
  Код (Key): 101 -> Студент (Value): Олена Коваль
  Код (Key): 102 -> Студент (Value): Андрій Петренко
  Код (Key): 103 -> Студент (Value): Марія Шевченко

--- 4. Виконання індивідуального Варіанту №7 (Облік фільмів) ---
Повний список кінофільмів у базі даних List:
Фільм: Початок (Inception)        | Жанр: Фантастика    | Рік: 2010 | Рейтинг: 8,8
Фільм: Інтерстеллар               | Жанр: Драма         | Рік: 2014 | Рейтинг: 8,7
Фільм: Темний лицар               | Жанр: Екшн          | Рік: 2008 | Рейтинг: 9,0
Фільм: Матриця                    | Жанр: Фантастика    | Рік: 1999 | Рейтинг: 8,7
Фільм: Кримінальне чтиво          | Жанр: Кримінал      | Рік: 1994 | Рейтинг: 8,9

Фільм із найвищим рейтингом за Варіантом №7:
  --> Фільм: Темний лицар               | Жанр: Екшн         | Рік: 2008 | Рейтинг: 9,0

Множина унікальних жанрів у базі Set:
  [Екшн, Фантастика, Кримінал, Драма]
```

Рисунок 5.1 – Вигляд терміналу після успішного виконання всіх чотирьох розділів Лабораторної роботи №4

На основі отриманих даних терміналу (рис. 5.1) проведено детальний покро-

ковий аналіз результатів перевіірки колекцій:

1) **Тестування динамічних списків (`List<Student>`):** Програма успішно ініціалізувала колекцію об'єктами трьох студентів. На основі методу `size()` було автоматично вираховано точний середній бал навчальної групи, який склав `91.27`. Порядок додавання елементів зберігся без викривлень.

2) **Тестування захисту від дублювання (`Set<String>`):** При спробі повторного надсилання ідентифікатора `"Java"` внутрішній механізм `HashSet` заблокував створення копії. У консоль було виведено чисту множину з трьох унікальних дисциплін, що повністю підтвердило закладені в теорії правила унікальності.

3) **Тестування індексної таблиці відображень (`Map`):** Впровадження структури `HashMap` дозволило виконати швидкий пошук за ключем-кодом `102`, миттєво повернувши об'єкт рядка `"Андрій Петренко"`. Повний перебір пар через `entrySet()` продемонстрував безпомилковий зв'язок між первинними ключами та значеннями.

4) **Аналіз індивідуального завдання Варіант №7 (Кінокаталог фільмів):** Сервісний менеджер `MovieManager` успішно проітерував список `List<Movie>`. Програма безпомилково розрахувала лідера рейтингу — фільм **«Темний лицар»** із максимальною оцінкою у **`9.0`** балів. Паралельно, завдяки структурі `Set<String>`, було автоматично сформовано та виведено унікальну множину жанрів: `[Фантастика, Кримінал, Драма, Екшн]`. Дублікат жанру «Фантастика» для фільму «Матриця» було успішно відфільтровано «на льоту».

6 ВІДПОВІДІ НА КОНТРОЛЬНІ ЗАПИТАННЯ

1. Що таке колекція в мові Java?

Колекція в Java — це спеціальний об'єкт, який служить контейнером і призначений для компактного зберігання, динамічного опрацювання та управління групою інших об'єктів. Колекції входять до складу *Java Collections Framework* (пакет `java.util`).

2. Для чого використовуються колекції?

Колекції використовуються для ефективної роботи з динамічними наборами даних у пам'яті комп'ютера, коли заздалегідь невідома точна кількість елементів. Вони надають готові високоефективні інструменти (алгоритми) для швидкого додавання, видалення, сортування, фільтрації та пошуку об'єктів.

3. Чим колекція відрізняється від масиву?

Масив має строго фіксований розмір, який не можна змінити після створення, і може зберігати як примітиви, так і об'єкти. Колекція може динамічно розширюватися або стискатися в процесі роботи програми, має великий вбудований набір методів обробки і здатна зберігати **виключно об'єктні типи даних**.

4. Яке призначення інтерфейсу `List`?

Інтерфейс `List` призначений для представлення **впорядкованої послідовності елементів** (динамічного масиву або списку). Головні особливості списків: кожен елемент має свій чіткий індекс починаючи з нуля, а сама структура повністю допускає наявність однакових значень (дублікатів).

5. Які основні властивості `ArrayList`?

`ArrayList` — це реалізація інтерфейсу `List` на базі звичайного масиву, який автоматично збільшує свою ємність при заповненні. Він забезпечує дуже швидкий доступ до елементів за їхнім індексом за константний час $O(1)$, але працює повільніше при видаленні чи вставці елементів всередину списку

через необхідність зсовування сусідніх комірок.

6. Для чого використовується метод `add()`?

Метод `add()` використовується для динамічного додавання нового елемента в кінець поточної колекції. У випадку інтерфейсу `List` цей метод завжди повертає `true` і збільшує розмір списку на одиницю.

7. Для чого використовується метод `get()`?

Метод `get(int index)` використовується виключно у списках `List` для миттєвого отримання посилання на об'єкт за його порядковим номером (індексом). Якщо вказати індекс, який виходить за межі фактичного розміру колекції, програма видасть помилку `IndexOutOfBoundsException`.

8. Яке призначення методу `set()`?

Метод `set(int index, E element)` призначений для заміни (перезапису) існуючого елемента списку за вказаним індексом на новий об'єкт. Метод повертає те старе значення, яке було замінене в цій комірці.

9. Чим відрізняються `remove()` та `clear()`?

Метод `remove()` видаляє з колекції **один конкретний елемент** (за його індексом або за значенням) і зменшує розмір структури на одиницю. Метод `clear()` повністю очищає колекцію, видаляючи абсолютно всі елементи одночасно й скидаючи розмір до нуля (`size = 0`).

10. Для чого використовується метод `contains()`?

Метод `contains(Object o)` використовується для перевірки наявності конкретного об'єкта всередині колекції. Він запускає внутрішній цикл порівняння елементів і повертає логічне значення `true`, якщо об'єкт знайдено, або `false`, якщо він відсутній.

11. Що повертає метод `size()`?

Метод `size()` повертає поточну кількість елементів (або пар «ключ – значе-

ння» для Map), які фактично містяться в колекції на момент виклику. Повертає значення примітивного типу int.

12. Для чого використовується `isEmpty()`?

Метод isEmpty() призначений для швидкої перевірки колекції на наявність елементів. Він повертає логічне значення true, якщо в колекції немає жодного елемента (тобто size() == 0), або false, якщо колекція містить хоча б один об'єкт.

13. Що таке `Set`?

Set (множина) — це інтерфейс *Java Collections Framework*, який описує колекцію, призначену для зберігання виключно **унікальних елементів**. На відміну від списків, множини не підтримують індексацію елементів, а порядок їхнього збереження (для HashSet) не гарантується.

14. Чим `Set` відрізняється від `List`?

Головна відмінність полягає в унікальності: List дозволяє зберігати дублікати й суворо упорядковує елементи за індексами, тоді як Set повністю блокує появу однакових значень. Крім того, у списку можна отримати елемент за номером за константний час через get(i), а в множині пошук здійснюється ітераційно або за хеш-кодом.

15. Чому `HashSet` не допускає дублювання елементів?

Внутрішній устрій HashSet базується на роботі асоціативної карти HashMap, де додані елементи стають ключами. При спробі додати елемент, HashSet розраховує його hashCode() та перевіряє вміст через метод equals(). Якщо такий об'єкт уже існує, нове значення просто перезаписує старе за тим самим хеш-індексом, запобігаючи подвоєнню.

16. Що таке `Map`?

Map (карта/відображення) — це інтерфейс, який представляє структуру даних для зберігання елементів у вигляді **пар «ключ – значення»**. Map не

успадковує базовий інтерфейс `Collection`, оскільки оперує унікальними ключами, кожному з яких відповідає строго одне певне об'єктне значення.

17. Що таке ключ і значення у структурі `Map`?

– **Ключ (*Key*):** унікальний об'єктний ідентифікатор, який служить «первинним ключем» (як у базах даних) для миттєвого пошуку пов'язаного об'єкта. Дублікати ключів заборонені.

– **Значення (*Value*):** дані (об'єкт), які прив'язані до конкретного ключа і витягуються за його запитом. Значення можуть дублюватися в різних ключах.

18. Для чого використовується метод `put()`?

Метод `put(K key, V value)` призначений для додавання нової пари «ключ – значення» до структури `Map`. Якщо операція виконується успішно, метод повертає `null`.

19. Що відбувається при додаванні нового значення за вже існуючим ключем?

Оскільки ключі в `Map` зобов'язані бути унікальними, при спробі викликати `put()` з уже існуючим у таблиці ключем, старе об'єктне значення за цим індексом буде **повністю стерте і замінене новим** значенням, що передане як аргумент. При цьому метод `put()` поверне посилання на те старе значення, яке щойно було видалене.

20. Для чого використовується `containsKey()`?

Метод `containsKey(Object key)` використовується для швидкої перевірки наявності конкретного ключа-індексу в асоціативній карті. Він повертає `true`, якщо такий ключ зареєстрований, або `false`, якщо ключ відсутній, що дозволяє уникнути помилки NullPointerException перед викликом `get()`.

21. Для чого використовується `entrySet()`?

Метод `entrySet()` використовується для отримання динамічної множи-

ни всіх записів відображення у вигляді `Set<Map.Entry<K, V>>`. Кожен елемент такої множини є повноцінним об'єктом пари, що дозволяє в одному циклі `for-each` одночасно й високоефективно витягувати як ключ через `getKey()`, так і пов'язане з ним значення через `getValue()`.

22. Що таке `generics`?

`Generics` (узагальнення / параметризовані типи) — це спеціальний архітектурний механізм мови Java, який дозволяє проектувати класи, інтерфейси та методи з параметрами, що виступають у ролі типів даних. Конкретний тип об'єктів підставляється розробником безпосередньо під час оголошення екземпляра.

23. Для чого використовуються `generics` під час роботи з колекціями?

Узагальнення використовуються для забезпечення **суворої типобезпеки (*Type Safety*)** на етапі компіляції додатка. Вони гарантують, що в колекцію не зможе потрапити сторонній об'єкт невідповідного типу, а також повністю позбавляють розробника необхідності виконувати небезпечне ручне приведення типів даних при витягуванні об'єктів із контейнера.

24. Що означає конструкція `List<String>`?

Дана конструкція означає параметризований динамічний список, який жорстко обмежений на роботу **виключно з об'єктами класу `String`**. Спроба додати туди число чи будь-який інший тип даних буде негайно заблокована компілятором Java, видавши помилку синтаксису.

25. Що означає конструкція `Map<Integer, Student>`?

Ця конструкція описує структуру асоціативної карти (відображення), в якій як унікальні ключі-ідентифікатори використовуються об'єкти класу-обгортки `Integer` (наприклад, ID або код студента), а як пов'язані з ними значення зберігаються реальні об'єкти користувацького класу `Student`.

26. У яких випадках доцільно використовувати `List`?

Списки `List` доцільно використовувати тоді, коли для програми критично важливо суворо **зберігати початковий порядок додавання елементів**, мати можливість швидкого доступу до об’єктів за їхніми порядковими номерами (індексами), а також коли бізнес-логіка повністю допускає наявність повторюваних значень (дублікатів).

27. У яких випадках доцільно використовувати `Set`?

Множини `Set` є незамінними в задачах, де ключовою вимогою є **гарантія унікальності елементів**. Їх доцільно обирати для фільтрації даних, виключення появи дублікатів (наприклад, формування унікального переліку жанрів кіно чи назв категорій товарів) та швидкої перевірки наявності об’єкта через метод `contains()`.

28. У яких випадках доцільно використовувати `Map`?

Структуру `Map` доцільно використовувати у випадках, коли необхідно організувати швидкий пошук об’єктів за принципом **«первинного ключа» (Primary Key)**. Вона ідеально підходить для проектування індексних таблиць, телефонних довідників, систем авторизації або логічного зв’язування унікальних ідентифікаторів (ID) з великими об’єктами сутностей.

7 ІНФОРМАЦІЙНІ РЕСУРСИ ТА ПОСИЛАННЯ НА ВИХІДНИЙ КОД

Згідно з вимогами до здачі лабораторної роботи, повний проект розробленого вихідного коду додатку, розміщено у віддаленому репозиторії системи контролю версій.

– **Посилання на Git-репозиторій проєкту:**
`https://github.com/nasuehfw/oop-lab4.git`

– **Електронний каталог спільного доступу:**
https://drive.google.com/drive/folders/1oiJNrP62_ZFaXsxSDoPd4y1sXu0Bt5s1?usp=drive_link